

Cahier des Charges R&D : Besoins Matériels, Logiciels & APIs pour l'Évolution du Projet BMCI

Ce document dresse la liste exhaustive des ressources requises (matériel GPU, abonnements cloud, clés d'API et infrastructures) pour passer du prototype actuel de simulation vocale à une plateforme industrielle performante et robuste. Une section spécifique détaille les prérequis nécessaires à l'intégration du modèle de Speech-to-Speech natif **Moshi** (Kyutai).

1. Besoins Matériels (Workstations & Serveurs GPU)

Option A : Pour le développement local et le Fine-Tuning

Pour éviter de dépendre de Google Colab/Kaggle et permettre d'entraîner et d'exécuter des modèles locaux (SLMs comme *Qwen 2.5 1.5B/3B* ou *TinyLlama 1.1B*) de manière fluide :

- **Station de travail R&D (Workstation Windows/Linux) :**
 - **GPU :** NVIDIA RTX 4080 (16 Go VRAM) ou RTX 4090 (24 Go VRAM).
L'architecture CUDA d'NVIDIA est indispensable pour PyTorch, Hugging Face, et l'accélération locale.
 - **RAM :** 32 Go ou 64 Go de RAM DDR5.
 - **Stockage :** 2 To SSD NVMe (pour le stockage des datasets, des checkpoints intermédiaires de modèles et des fichiers audio de test).
 - **Processeur :** Intel Core i7/i9 ou AMD Ryzen 7/9 (minimum 8 cœurs physiques).

Option B : Pour l'hébergement de production dans le Cloud

- **Instance GPU Dédiée (AWS, Google Cloud, RunPod ou Scaleway) :**
 - **Configuration minimale :** Instance munie d'une carte NVIDIA A10G ou L4 (24 Go de VRAM).
 - **Configuration recommandée (pour Moshi ou plusieurs utilisateurs simultanés) :** Instance munie d'une NVIDIA A100 (40 Go/80 Go) ou H100.
-

2. Besoins en APIs Cloud & Augmentation des Quotas (Abonnements payants)

Pour supprimer les limites de requêtes par minute (HTTP 429 Too Many Requests) et garantir la disponibilité de la plateforme vocale en direct :

Fournisseur	Rôle dans le pipeline	Type d'accès requis	Justification technique
-------------	-----------------------	---------------------	-------------------------

Mistral AI	LLM (mistral-small-latest) & TTS (Voxtral Marie)	Compte payant (Pay-as-you-go) avec quotas élevés	Utilisation intensive de la synthèse vocale émotionnelle native Voxtral sans interruption de service.
Cohere	STT (Transcription vocale ultra-rapide)	Compte payant (Production tier)	Transcription en direct des stagiaires avec un taux d'erreur de mots (WER) minimal.
Hume AI	TTS alternatif (Voix ultra-naturelle et expressive)	Abonnement Pro / Pay-as-you-go	Alternative au TTS Mistral si l'on souhaite intégrer des voix avec analyse émotionnelle en temps réel.
ElevenLabs	TTS & STT de secours	Abonnement Starter / Creator	Accès à des voix clonées haut de gamme si besoin d'élargir le catalogue de clients fictifs.

3. Besoins d'Infrastructure Réseau & Hébergement

- **Abonnement LiveKit Cloud (ou Serveur Dédié) :**
 - Actuellement, le projet tourne sur une instance de bac à sable gratuite (sandbox).
 - **Recommandation :** Passer sur un abonnement **LiveKit Cloud payant** (pour garantir la bande passante WebRTC et la réduction de gigue audio) OU déployer un serveur **LiveKit open-source** auto-hébergé sur une machine virtuelle Linux (Ubuntu/Debian) dans le Cloud de la BMCI.
- **Passerelle de Téléphonie (SIP / VoIP) :**
 - Indispensable si la BMCI souhaite que les stagiaires s'entraînent en appelant l'IA avec un vrai téléphone ou softphone.
 - **Outils requis :** Abonnement à un service Trunk SIP (ex: Twilio, Telnyx) couplé à une passerelle SIP/WebRTC LiveKit.

4. Spécificités Techniques pour l'Intégration de MOSHI (Kyutai)

Si le projet décide de migrer vers **Moshi** (le modèle de Speech-to-Speech natif ultra-rapide < 200ms de latence de Kyutai), voici les besoins et verrous techniques spécifiques à adresser :

1. Prérequis Matériels stricts (Serveur GPU requis)

- **Pas de CPU :** Moshi ne peut pas s'exécuter en temps réel sur CPU.
- **VRAM requise :** Le modèle complet (Helium 7B + Mimi codec) requiert obligatoirement un serveur équipé d'un GPU professionnel avec **minimum 16 à 24 Go de VRAM dédiée rapide** (comme les serveurs NVIDIA A10G, A100 ou L4).

2. Environnement Système (OS)

- **Système d'exploitation :** Serveur sous **Linux (Ubuntu 22.04 LTS de préférence)**. Kyutai a conçu Moshi pour Linux et macOS (Apple Silicon).

- **Développement sous Windows** : Si les tests doivent se faire sous Windows, il est obligatoire d'installer **WSL2 (Windows Subsystem for Linux)** et de configurer le partage des pilotes GPU CUDA entre Windows et le conteneur Linux.

3. Ressources de Développement & Transport Audio

- **Développement d'un client de codec audio** : Moshi nécessite de compresser et décompresser l'audio à la volée avec son codec propriétaire **Mimi**. Il faut donc développer ou intégrer un SDK client (TypeScript, Kotlin, Swift ou Flutter) capable de capturer le micro, encoder l'audio avec Mimi, l'envoyer en streaming via WebSockets, et décoder le flux retourné par Moshi.
- **Détection vocale et purge de buffers (VAD)** : Pour permettre au stagiaire d'interrompre l'IA en parlant, vous devez implémenter un détecteur de silence et de parole local. Dès que l'utilisateur commence à parler, le client doit envoyer un signal d'interruption instantané au serveur Moshi pour vider le tampon audio et forcer le modèle à se taire immédiatement.

4. Interfaçage avec les APIs métier (Function Calling)

- Moshi génère sa voix et son monologue intérieur en flux tendu, mais il ne sait pas exécuter de fonctions. Pour intégrer des scénarios complexes (ex: bloquer une carte bancaire), il faut développer un **analyseur de flux de texte en arrière-plan** qui détecte les intentions de l'IA, interroge l'API BMCI et réinjecte la réponse sous forme de contexte textuel dans Moshi pour adapter sa phrase vocale suivante.